MATHEMATICS / INDEPENDENT STUDY

MM8036

Type Theory

Study handbook and reference

Original teaching material aligned with Stockholm University course descriptions.
Independently authored; not an official university publication.

Edition 1 · 24 October 2026

Contents

1 A route through type theory 3
1.1 Official information and supplementary references 3
2 Diagnostic and logic bridge 4
2.1 Answers and routing 4
2.2 Logic as rules for evidence 4
3 Unit study guides 6
3.1 U1 · Binding and substitution 6
3.2 U2 · Reduction and confluence 6
3.3 U3 · Simple types and judgments 6
3.4 U4 · Normalization and constructive logic 7
3.5 U5 · Dependent functions and pairs 7
3.6 U6 · Induction, identity, and universes 7
3.7 U7 · Substitution and meaning 8
3.8 U8 · Categorical models and formal proofs 8
4 Checkpoint answers and cumulative revision 9
4.1 Retrieval schedule and proof strategy cards 9
4.2 Mock assessment and self-assessment 9
4.3 Oral explanation practice 10
5 Optional Agda laboratories 11
5.1 Lab 1 · Proofs as programs 11
5.2 Lab 2 · Induction and equality 11
5.3 Lab 3 · Dependent data 12
5.4 Lab 4 · Equality and transport 12
6 Notation and Swedish–English glossary 13

1 A route through type theory

This course moves from the syntax of functions to formal evidence and mathematical models of that evidence. The central habit is to keep three questions separate: Is this expression well formed? Does it have the required type? What happens when it computes? A fourth question, whether an interpretation validates a judgment, belongs to semantics. Confusing these questions causes more difficulty than the notation itself.

Study each unit in four sessions of roughly 60–90 minutes, extending these when needed: read the first lecture and reconstruct one proof; read the second and work its examples without looking; solve E and selected C problems; return after a day for retrieval and A problems. The normalization proof deserves several sessions. Optional labs follow Units 4, 6, 6, and 8; all core mathematical content is readable without running software.

For each proof, keep a "judgment ledger": context, proposed term, proposed type, rule used, and any computation needed to match the rule. If a proof fails, find the first line whose premises have not been established. Do not try to repair a type mismatch by repeatedly rewriting the final expression without checking its context.

E/C/A labels describe increasing demands within university mathematics, not SU grading rules. E requires accurate definitions and justified basic steps; C requires several connected decisions; A requires synthesis, generalization, or a carefully justified counterexample. The four E, five C, and three A exercises in each unit are followed by two cumulative practice assessments.

1.1 Official information and supplementary references

Course baseline: Stockholm University catalogue, MM8036 Type Theory and the English syllabus version valid from autumn 2013, consulted 24 October 2026. The course is 7.5 credits; the catalogue's logic prerequisite motivates the explicit bridge below. The eight teaching units in this package are an independently designed sequence, not an official SU timetable.

The syllabus topics are mapped as follows: lambda calculus and substitution to U1; reduction and confluence to U2; simple types and derivations to U3; normalization and constructive logic to U4; Martin-Löf dependent types to U5–6; meaning explanations and explicit substitutions to U7; categorical interpretations and proof-assistant applications to U8. Every topic is practiced on its same-numbered exercise sheet. Proof construction is assessed throughout; explanation and comparison appear in the A problems and oral practice. See the root coverage map for cross-course dependencies.

Supplementary mathematical reading: Per Martin-Löf, *Intuitionistic Type Theory* (1984), for the constructive perspective; PLFA, Lambda and PLFA, Properties, for examples of syntax and metatheoretic proofs. PLFA uses a richer language in places; its recursion must not be silently imported into our arrow-only normalization theorem. These are supplementary sources, distinct from official course information. The exposition, exercises, and solutions here are independently authored.

Software references: Agda command-line documentation, safe mode, and without-K mode, consulted 24 October 2026. The actual pinned executable is Agda 2.8.0.2, not a promise to track the newest release. Its version and download checksum are recorded in the project tooling manifest.

2 Diagnostic and logic bridge

Attempt these before Lecture 1. A missed item is a routing signal, not a grade.

1. Negate "for every natural number there exists a larger natural number," retaining the quantifier order.
2. Give a direct proof of $P \to (Q \to P)$ using assumptions and discharge.
3. Explain why proving $P \to Q$ does not prove $Q \to P$.
4. State structural induction for a syntax with variables, binary application, and abstraction.
5. Distinguish a function from a set of inputs to outputs from the expression naming one of its values.
6. Explain why an existential proof should identify its witness when working constructively.
7. Show why a finite expression cannot equal a strictly larger expression obtained by adding an outer constructor around it.
8. Identify the invalid step: "Every term has at most one normal form; therefore every term has a normal form."

2.1 Answers and routing

1. In classical quantifier notation the negation is "there exists n such that every m satisfies $m \leq n$." Negation changes both quantifiers and negates the comparison. For intuitionistic reasoning, do not assume every classical transformation is a constructive equivalence; this diagnostic tests the quantifier reading. Revisit the evidence interpretation below before Unit 4.

2. Assume $p : P$, then assume $q : Q$; the original p still establishes P. Discharge q, then p. The term is $\lambda p.\lambda q.p$. Review introduction rules below if the order of discharge is unclear.

3. Even numbers being integers does not make every integer even. In a proof of $P \to Q$ the starting evidence has type P, not Q. Review implication elimination.

4. Prove the property for each variable; prove it for an application from the property for its two children; prove it for an abstraction from the property for its body. These cover exactly the constructors. Review recursive definitions in Lecture 1.

5. A function f is the whole assignment, while $f(a)$ is its output at one input. This distinction reappears in function types, currying, and dependent functions. Review arrows before U3.

6. A constructive existential proof packages a witness a and evidence of the predicate at a. "A witness cannot fail to exist" does not necessarily provide this package. Review Sigma types in U5.

7. Count constructor nodes: adding an outer arrow node and its second subtree strictly increases the count. This simple invariant rules out the finite simple-type equation $A = A \to B$. Review U3.09.

8. "At most one" allows zero. Confluence concerns compatibility of paths; termination is a separate question. Review U2 before attempting the normalization argument.

2.2 Logic as rules for evidence

An introduction rule explains how to build evidence; an elimination rule explains how to use it. To prove $A \to B$, assume a named $a : A$ and construct evidence of B, then discharge the assumption. To use $A \to B$, also supply evidence of A. An implication alone does not produce its conclusion. A context lists which evidence is available; bound variables are the names of discharged assumptions.

$$\frac{\Gamma, a : A \vdash t : B}{\Gamma \vdash \lambda a.t : A \to B}$$

Implication introduction / abstraction

$$\frac{\Gamma \vdash f : A \to B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B}$$

Implication elimination / application

Conjunction introduces a pair and eliminates by projection. Disjunction introduces a chosen tag and eliminates by proving a common target from each possible tag. Falsehood has no constructor; an assumed impossible input may be eliminated. Universal evidence accepts an arbitrary input. Existential evidence includes a particular witness. These interpretations explain the corresponding type constructors; they do not license a global excluded-middle decision procedure.

Worked proof repair. To prove $(A + B) \to C$, a student supplies only $f : A \to C$. The right-input branch has no result of type C. The repair is either to add $g : B \to C$, or strengthen the hypothesis to exclude the right branch with a proof of $B \to 0$. One must repair the missing evidence, not merely add a symbol to the term.

Induction choices. Induct on syntax to prove a substitution property. Induct on a typing derivation to prove weakening or preservation. Induct on a reduction sequence to lift a one-step theorem to many steps. Induct on a natural number or vector to construct evidence indexed by that datum. Write the induction object and hypothesis before the cases.

3 Unit study guides

3.1 U1 · Binding and substitution

Objectives U1.1–U1.2: read syntax and alpha equivalence; perform and reason about capture-avoiding substitution. Read Lectures 1–2, then the U1 sheet.

Worked example. In $(\lambda x.\lambda y.xz)[z := xy]$, first rename both binders, for instance to a, b. The result is $\lambda a.\lambda b.a(xy)$. The originally free variables in the argument remain free. The original bound occurrence of x became a; it must not be replaced as though it were the free x in the supplied argument.

Study move: draw one line from each bound occurrence to its binder, then erase the names mentally. **Proof repair:** a claimed substitution formula without freshness conditions should be tested on a one-variable term first. **Retrieval:** explain why alpha renaming preserves node count but does not allow arbitrary renaming of free variables.

3.2 U2 · Reduction and confluence

Objectives U2.1–U2.2: compare full reduction and evaluation strategies; use confluence correctly. Read Lectures 3–4.

Worked example. For $(\lambda x.z)((\lambda y.y)w)$, outer-first reduction yields z. Argument-first gives $(\lambda x.z)w$ and then z. The fork joins even though one path discards work. If the argument were Ω, an outer path still terminates while reducing only the argument diverges. This illustrates why confluence is not a speed or termination theorem.

Study move: draw a fork with a common source and fill in the two joining paths. **Proof repair:** normal-form uniqueness requires both confluence and normality of the endpoints. **Retrieval:** distinguish a value under weak evaluation from a full beta normal form.

3.3 U3 · Simple types and judgments

Objectives U3.1–U3.2: construct derivations; prove weakening, substitution, preservation, and closed progress. Read Lectures 5–6.

Worked example. To type $\lambda f.\lambda x.f(fx)$, choose $x : A$. The first application requires $f : A \to A$ if its output is to be used as another input to the same f. The resulting type is $(A \to A) \to A \to A$. A derivation checks consistency of repeated variable uses, rather than assigning each occurrence a fresh unrelated type.

Study move: infer types from the leaves upward and discharge assumptions from the inside outward. **Proof repair:** preservation at a root redex uses inversion to expose the body judgment, then substitution. **Retrieval:** give a typed open stuck term and identify the exact closedness hypothesis it violates.

3.4 U4 · Normalization and constructive logic

Objectives U4.1–U4.2: reconstruct the reducibility proof; read proofs as programs. Read Lectures 7–8 and optionally Lab 1.

Worked example. The term $\lambda x.xx$ is normal as raw syntax, but applying it to itself diverges. The arrow reducibility predicate excludes this failure by quantifying over reducible arguments. The fundamental lemma handles the abstraction case by first showing the body terminates with a fresh variable, and then showing its substituted body is reducible for an arbitrary reducible argument. Controlled beta expansion connects those two facts to the application of the abstraction.

Study move: reconstruct the dependency chain: candidate properties ⟶ controlled expansion ⟶ fundamental lemma ⟶ strong normalization. **Proof repair:** do not claim this proof establishes normalization of unrestricted recursion, all dependent types, or every Agda feature. **Retrieval:** build a program for constructive contraposition without invoking excluded middle.

3.5 U5 · Dependent functions and pairs

Objectives U5.1–U5.2: maintain context dependencies; construct and eliminate Pi and Sigma types. Read Lectures 9–10.

Worked example. An input $h : \Pi_{n:\mathrm{Nat}}\Sigma_{v:\mathrm{Vec}(A,n)}P(n,v)$ supplies, at each length, both a vector and evidence about it. First projections define $f(n) : \mathrm{Vec}(A,n)$; second projections define evidence of $P(n, f(n))$. The type of the second projection is correct because the definition of f computes to that first projection.

Study move: underline every occurrence of the index in a type, then check that every eliminator substitutes the actual index. **Proof repair:** two existential witnesses from different sums cannot automatically be combined into one common witness. **Retrieval:** distinguish pointwise computation rules from a theorem of identity between functions.

3.6 U6 · Induction, identity, and universes

Objectives U6.1–U6.2: formulate motives and dependent recursion; use J, transport, and universe levels. Read Lectures 11–12 and optionally Labs 2–3.

Worked example. To compare two vector expressions of lengths $m + n$ and $n + m$, an equality proof of the lengths does not make the displayed result types definitionally identical. Transport one vector along the length equality before asking for an identity between vectors in the same fiber. In contrast, append's own clauses match its result index directly by the defining equations of addition.

Study move: reduce the target type before selecting reflexivity. If reduction stops at a variable, look for an induction principle. **Proof repair:** the motive of J must be valid while its endpoint varies. **Retrieval:** explain why $\Pi_{A:U_0}(A \to A)$ belongs to U_1.

3.7 U7 · Substitution and meaning

Objectives U7.1–U7.2: manipulate typed substitution tuples; interpret contexts as sets and types as families. Read Lectures 13–14.

Worked example. Let the context be $n : \mathrm{Nat}, v : \mathrm{Vec}(D, n)$. A substitution from the empty context may supply the numeral two and a closed two-element vector. It cannot supply a three-element vector with the same first component. In the set model the context is a total space of fibers, and a closed substitution chooses one point of that total space. The typing condition is exactly membership in the correct fiber.

Study move: draw source and target of every substitution before composing it. **Proof repair:** typed equational laws alone do not imply that every orientation of them terminates as a rewrite system. **Retrieval:** explain why soundness cannot be used backwards to import UIP from the set model.

3.8 U8 · Categorical models and formal proofs

Objectives U8.1–U8.2: use products, exponentials, and comprehension; audit a checked proof against its mathematical statement. Read Lectures 15–16 and optionally Lab 4.

Worked example. A body interpreted by $h : \Gamma \times A \to B$ becomes a function term by currying. Applying it to $u : \Gamma \to A$ evaluates the curry at $(\gamma, u(\gamma))$, giving $h(\gamma, u(\gamma))$. This is also the interpretation of substituting u into the body's added variable, so beta soundness is a direct interaction of currying and substitution.

Study move: write the universal property's existence and uniqueness clauses before drawing its diagram. **Proof repair:** CCC structure handles simple arrows; dependent families require additional reindexing and comprehension structure. **Retrieval:** explain the difference between an unfilled goal, a postulate, and a constructed proof accepted under safe mode.

4 Checkpoint answers and cumulative revision

1.1: $\{y, z\}$; the first z is free and the inner z is bound. **1.2:** rename first, obtaining $\lambda z.yz$.

2.1: an abstraction is a weak call-by-value value, even if its body diverges under full reduction; $\lambda z.\Omega$ is not beta normal. **2.2:** confluence compares paths and does not assert that one ends.

3.1: self-application would force the impossible finite-type equation $A = A \to B$. **3.2:** an open variable is typable but is neither a value under the lecture convention nor able to step.

4.1: the arrow predicate controls every reducible argument, which is exactly what the application induction case needs. **4.2:** a sum contains a selected tag and evidence for that selected side.

5.1: a vector declaration depends on the previously declared length. **5.2:** the second projection lies in the fiber selected by the first projection.

6.1: addition inspects its first argument; zero exposes a constructor and a variable does not. **6.2:** transport computes directly on reflexivity; a variable path must be eliminated before that clause applies.

7.1: the final context is Θ, with composite $\sigma \circ \rho$. **7.2:** a term is a section taking each environment to an element of its appropriate fiber.

8.1: dependent types require indexed families and coherent substitution/comprehension beyond fixed exponentials. **8.2:** a hole is an unsupplied term, so parsing the surrounding text does not discharge its proof obligation.

4.1 Retrieval schedule and proof strategy cards

After U2, perform a fresh capture-avoiding substitution and draw a nontrivial joining diagram. After U4, reconstruct preservation and explain why normalization needed a stronger invariant. After U6, type a dependent projection, write an induction motive, and derive transport. After U8, interpret one beta computation in sets and audit a lab signature. Return to each prompt after one day, one week, and after a mock assessment; record whether the obstacle was a definition, a hypothesis, or a proof idea.

Use these strategy cards when stuck: a syntax property suggests structural induction; a theorem about typing suggests induction on derivations and inversion; a theorem about arbitrary dependent data suggests its eliminator with an explicit motive; a uniqueness claim suggests a universal property; an overstrong claim suggests a minimal counterexample such as a variable, an empty fiber, or an ignored divergent argument. Choose a card before reading Hint 1.

4.2 Mock assessment and self-assessment

Each mock has two E, two C, and two A problems and a suggested total time of 180 minutes. A useful allocation is 15 minutes per E problem, 30 per C, 40 per A, and 10 minutes to review. These times are independent of the unit practice estimates. Give each solution 0–4 feedback points: 0 no usable start; 1 relevant definitions but a major gap; 2 a correct route with an unresolved typing or proof step; 3 a complete argument with minor presentation defects; 4 a complete, precise, well-explained argument. This rubric measures practice quality and makes no claim about official university grade thresholds.

When reviewing, identify the first invalid line, repair it, and solve a nearby variant without the solution. A numerical score without a diagnosis gives little guidance about the next study session.

4.3 Oral explanation practice

Give each response in two minutes, then answer the challenge in another minute.

1. Explain capture avoidance. Challenge: why is $\lambda y.y$ the wrong result of $(\lambda y.x)[x := y]$? **Answer:** it captures a formerly free variable; a fresh binder must leave the inserted y free.
2. Distinguish confluence from termination. Challenge: give one system with confluence and divergence. **Answer:** Ω in lambda calculus, or $a \to a, a \to b$ with b normal, provides the required divergence without incompatible normal forms.
3. Explain substitution's role in preservation. **Answer:** inversion exposes a body typed with a bound assumption and an argument of that assumption's type; substitution removes the assumption while preserving the result type.
4. Explain reducibility to a student who knows termination. **Answer:** at arrow types it additionally controls every reducible application, repairing the otherwise failed application induction case.
5. Distinguish Pi from Sigma. **Answer:** Pi accepts arbitrary indices and returns evidence; Sigma supplies one index with evidence in its fiber.
6. Explain judgmental versus propositional equality. **Answer:** judgmental equality supports conversion by the theory's computation rules; identity evidence is an internal term consumed by J or transport.
7. Explain semantic soundness. **Answer:** derivations remain valid after interpretation; the converse is an additional theorem and cannot be presumed.
8. Explain what a checked Agda proof certifies. **Answer:** acceptance at its written type under the recorded rules, which must still be compared with the intended theorem.

5 Optional Agda laboratories

The files are in `labs/starters` and `labs/solutions` beside this handbook. Each lab is self-contained, with no external standard-library dependency. Starter question marks deliberately mark tasks to complete; the completed solutions have no holes or postulates. The completed four labs were checked with project-local Agda 2.8.0.2 and `--safe --without-K`.

From the project root, run `./scripts/check-labs.ps1` to recheck completed solutions. It creates only project-local Agda data/configuration directories and uses the pinned executable. To work on a starter, copy the relevant starter to a separate working directory, retain its module filename, and use `./scripts/check-labs.ps1 -LabDirectory <your-working-directory>`. The checker will report unsolved goals until you replace the question marks with complete terms or clauses. Keep starter and solution directories as separate include paths because they intentionally use the same module names.

5.1 Lab 1 · Proofs as programs

After U4, implement `identity`, `constant`, `compose`, `swap`, `cases`, `absurd`, `distribute`, and `factor`. First write the corresponding proposition on paper. A function clause discharges assumptions; a product pattern makes both components available; a sum pattern creates two branches. The absurd pattern `()` states that an empty type has no constructor cases. Do not invent a value of the empty type.

Milestones: first check identity and composition, then product and sum elimination, then distributivity. **Self-check:** on each left/right constructor, trace the composite of distribute and factor to the original constructor data. The completed file gives the two maps; the paper computation explains their relationship.

5.2 Lab 2 · Induction and equality

After U6, complete `zero-right`, `associative`, and the concrete `two-plus-two` check. The supplied addition recurses on its first argument. Reduce each target on paper before choosing reflexivity or a recursive proof. For right-zero, the successor clause must lift the recursive equality by `cong suc`. For associativity, induct on the first addend for the same reason.

Milestones: explain why zero-left is immediate, prove zero-right, then generalize the same pattern to associativity. **Self-check:** every recursive call has a structurally smaller first argument. The finite numeral check is computation; the general laws are induction.

5.3 Lab 3 · Dependent data

After U6, implement vector `map`, `append`, `head`, `tail`, and `replicate`, then prove `map-identity` by vector induction. The input and output indices describe the intended behavior before code is written. In append, matching the first vector exposes exactly the constructor needed by the definition of addition. In head and tail, the successor index excludes nil. For `map-identity`, lift the recursive tail equality through the fixed-head constructor; Lecture 16 supplies the complete paper derivation.

Milestones: map preserves length, append combines lengths, head/tail require a successor length, and replicate constructs the requested length. **Self-check:** a sample vector has length two and appending it to itself has length four. Explain why changing append's signature to the reversed sum may require an explicit equality proof rather than the same clauses checking immediately.

5.4 Lab 4 · Equality and transport

After U8, complete symmetry, transitivity, congruence, transport, transport computation, transport composition, vector cast, and the cast reflexivity proof. Each equality proof is obtained by permitted path induction. The transport-composition theorem compares terms in the same final fiber; check its type carefully before writing clauses.

Milestones: solve symmetry and congruence, then transport and its reflexive computation, then composition and vector casting. **Self-check:** `cast refl v` computes to `v`; a general equality proof changes the index without editing the vector elements. The safe and without-K flags are part of the lab contract, not obstacles to switch off.

6 Notation and Swedish–English glossary

Notation	Meaning
$\Gamma \vdash t : A$	Term t has type A under context Γ.
$t[x := u]$	Capture-avoiding replacement of free occurrences of x.
$\underset{\beta}{\rightarrow}, \overset{*}{\rightarrow}$	One beta step; zero or more beta steps.
$\mathrm{FV}(t)$, SN	Free variables; strongly normalizing raw terms.
$\Pi_{x:A} B(x)$	Dependent function type.
$\Sigma_{x:A} B(x)$	Dependent pair type.
$\equiv$, $\mathrm{Id}_{A(a,b)}$	Judgmental equality; internal identity type.
U_i	Predicative universe at level i.
$\sigma : \Delta \to \Gamma$	Terms in Δ supplying declarations of Γ.
$\Gamma \cdot A$	Context extension/comprehension.

Swedish	English
bunden / fri variabel	bound / free variable
substitution utan variabelinfångning	capture-avoiding substitution
härledning	derivation
antagande / slutsats	assumption / conclusion
kontext / omdöme	context / judgment
konfluens / normalform	confluence / normal form
stark normalisering	strong normalization
beroende typ	dependent type
induktionsprincip	induction principle
identitetstyp	identity type
vittne	witness
bevisassistent	proof assistant
tolkning / sundhet	interpretation / soundness

Terminology varies across Swedish texts, especially for “judgment” and “definitional equality.” Use the accompanying rule or definition to resolve a translation; do not infer a new mathematical distinction from a different word alone.